

Experiment 4

Enhancing the Portfolio Website: Semantic Structure, Text Formatting, Links, Images, Tables and Forms

Aim

To extend the personal portfolio webpage built in the previous experiment (using only heading, paragraph, and list tags) by adding semantic structural elements, richer text-level markup, hyperlinks, images, a data table, and a working contact form — making it a complete, realistic, and well-structured portfolio page.

Software Required

- Visual Studio Code (or any text editor)
- Any web browser (Google Chrome / Microsoft Edge / Firefox)
- The `index.html` file created in the previous experiment

Prerequisite

Students must have their portfolio page from the last session ready, containing:

- One `<h1>` with their name
- One or more `<p>` tags describing themselves
- `<ul>/<ol>` `<li>` lists for skills, projects, and achievements

1 Theory

So far the portfolio page is just a flat sequence of headings, paragraphs, and lists. A browser can display it, but it has no structure — there is no way to tell which part is the header, which part is the main content, and which part is the footer, and the text itself carries no meaning beyond how it looks. Today's experiment introduces the tags that give a page real structure, the tags that add meaning to text itself, the tags that let a page link out to the rest of the web, the tag that lets a page show an image, the tags that present tabular data, and the tags that collect information from a visitor through a form. Every new tag is explained first, with a short generic example, before you apply it to your own portfolio in the Procedure section.

1.1 Semantic Structural Tags

“Semantic” tags are tags whose name itself describes the purpose of the content inside them, rather than just its visual appearance. Before HTML5, developers used generic containers for every section of a page, and the only way to know what one meant was to read a comment or a class name. Semantic tags remove that guesswork for both developers and browsers.

`<header>`

Represents the introductory part of the page — typically a title, tagline, and a navigation menu. A page normally has one `<header>`.

```
<header>
  <h1>My Site</h1>
  <p>Welcome to my page</p>
</header>
```

<nav>

Represents a block of navigation links (menu). Wrapping links in `<nav>` tells the browser “these links exist to help the user move around the page or site.”

```
<nav>
  <a href="#home">Home</a>
  <a href="#about">About</a>
</nav>
```

<main>

Represents the single, central content of the page — the part unique to this page, not repeated on every page as a header/footer would be. A page should have only one `<main>`.

```
<main>
  <p>The main content of this page.</p>
</main>
```

<section>

Represents a thematic grouping of content, usually with its own heading.

```
<section>
  <h2>Our Services</h2>
  <p>A short description.</p>
</section>
```

<article>

Represents a self-contained piece of content that would still make sense if pulled out and shown on its own.

```
<article>
  <h3>Blog Post Title</h3>
  <p>Post content goes here.</p>
</article>
```

<footer>

Represents the closing part of the page — copyright line, contact details, social links. A page normally has one `<footer>`.

```
<footer>
  <p>&copy; 2026 My Company</p>
</footer>
```

Semantic tags behave exactly like a generic block container on screen (they don't look different by default) — their value is in meaning, accessibility (screen readers announce them), and SEO (search engines use them to understand page content).

1.2 Text-Level Semantic Tags

Beyond page structure, HTML also has tags that add meaning to specific words or phrases within your text — not just visual styling.

`<strong>` / `<em>`

`<strong>` marks text as important; `<em>` marks text as stressed/emphasized. Both usually render as bold/italic, but carry real meaning for screen readers.

```
<p><strong>Note:</strong> submit the form
  <em>before</em> Friday.</p>
```

`<blockquote>` / `<q>`

`<blockquote>` sets off a long quoted block; `<q>` is for a short inline quote. Both accept a `cite` attribute linking to the quote's source.

```
<blockquote cite="/source">
  Design is how it works.
</blockquote>
<p>She said <q cite="/source">keep it simple</q>.</p>
```

`<cite>` / `<dfn>`

`<cite>` names a referenced work or source; `<dfn>` marks the defining instance of a term.

```
<cite>The Elements of Style</cite>
<p>A <dfn>variable</dfn> stores a value.</p>
```

`<abbr>`

Marks an abbreviation; the `title` attribute holds the full expansion, shown as a tooltip on hover.

```
<abbr title="HyperText Markup Language">HTML</abbr>
```

<address>

Wraps contact information for a page or article's author.

```
<address>Contact: info@example.com</address>
```

<ins> /

Marks text that was inserted or removed, useful for showing an edit or a price change.

```
<p>Price: <del>€8377;500</del> <ins>€8377;399</ins></p>
```

<sup> / <sub>

Raise or lower text as superscript or subscript.

```
<p>H<sub>2</sub>O and 4<sup>th</sup> place</p>
```

1.3 The Anchor Tag — <a>

The anchor tag creates a hyperlink — clickable text or an image that takes the user somewhere else. Its most important attribute is **href** (“hypertext reference”), which holds the destination.

External link

href points to another website.

```
<a href="https://github.com/username">GitHub</a>
```

target="_blank"

Opens the link in a new browser tab instead of navigating away from the current page. Pair it with **rel="noopener"** for security.

```
<a href="https://example.com" target="_blank"
  rel="noopener">Visit Site</a>
```

Email link

href="mailto:..." opens the user's default email app with the address pre-filled.

```
<a href="mailto:name@email.com">Email Me</a>
```

In-page link

href="#id" jumps to the element on the same page whose **id** matches. This is how a navigation menu scrolls to a section instead of loading a new page.

```
<a href="#projects">Projects</a>
```

Download link

The `download` attribute makes the browser save the linked file instead of navigating to it.

```
<a href="resume.pdf" download>Download Resume</a>
```

1.4 The Image Tag — `<img>`

The image tag embeds a picture into the page. Unlike every tag used so far, `<img>` is a self-closing / void tag — it has no closing tag and no content between tags, because an image cannot “contain” anything.

`src`

(“source”) the file path or URL of the image — required.

`alt`

(“alternative text”) a short text description of the image, shown if the image fails to load and read aloud by screen readers — required for accessibility.

`width / height`

optional attributes to control the displayed size, in pixels.

`loading="lazy"`

tells the browser to defer loading this image until it is about to scroll into view — improves page speed.

```

```

1.5 Grouping a Caption with an Image

The `<figure>` tag groups an image together with its caption (`<figcaption>`) as a single semantic unit.

```
<figure>
  
  <figcaption>Home screen of the app</figcaption>
</figure>
```

1.6 Enhancing Lists

Beyond a plain `<ul>/<ol>`, HTML lists support a few useful extras:

Nested lists

A list placed inside an `<li>` of another list, for sub-items or categories.

```
<ul>
  <li>Frontend
    <ul><li>HTML</li><li>CSS</li></ul>
  </li>
</ul>
```

type / start / reversed

Customize an `<ol>`'s numbering style, starting number, or counting direction.

```
<ol type="A" start="3" reversed>
  <li>Third item</li>
</ol>
```

`<dl>` `<dt>` `<dd>`

A definition list pairs a term (`<dt>`) with its description (`<dd>`) — useful for a glossary or FAQ.

```
<dl>
  <dt>API</dt>
  <dd>A way for programs to talk to each other</dd>
</dl>
```

1.7 The id and class Attributes

id

A unique label given to exactly one element on the page. Used as the target of in-page links (`href="#about"` needs an element with `id="about"`).

class

A label that can be given to many elements at once, used later to style or select a group of elements together with CSS.

```
<section id="about" class="page-section">
  <h2>About</h2>
</section>
```

1.8 Tables

A table presents information in rows and columns — use it for genuinely tabular data (a timetable, a price list, a skills matrix), never as a page-layout trick.

`<table>` `<tr>` `<td>`

The table container, a table row, and a plain data cell.

```
<table>
  <tr><td>Name</td><td>Asha</td></tr>
</table>
```

`<th>` + `scope`

A header cell, bold and centered by default; `scope="col"` or `scope="row"` tells assistive technology what the heading applies to.

```
<tr><th scope="col">Skill</th></tr>
```

`colspan` / `rowspan`

Merge a cell across multiple columns or rows.

```
<td rowspan="2">Frontend</td>
<td colspan="2">Total</td>
```

`<thead>` `<tbody>` `<tfoot>`

Groups a table into a header, body, and footer section for structure and styling.

```
<table>
  <thead><tr><th>Skill</th></tr></thead>
  <tbody><tr><td>HTML</td></tr></tbody>
</table>
```

1.9 Forms

A form collects information from the visitor and sends it somewhere — typically to a server, though in this experiment we will only build the front-end structure.

`<form>`

The `action` attribute sets where the data is sent; `method` sets how (GET or POST).

```
<form action="/submit" method="post">
  ...
</form>
```

<input>

A single-line control; its **type** attribute decides its behavior — **text**, **email**, **password**, **date**, **radio**, **checkbox**, **file**, and more.

```
<input type="email" name="visitor_email" required>
```

<textarea>

A multi-line text input; **rows**/**cols** set its visible height and width.

```
<textarea name="message" rows="4" cols="40"></textarea>
```

<select> <option>

A dropdown list of choices; the **selected** attribute pre-picks a default option.

```
<select name="topic">
  <option value="job" selected>Job Opportunity</option>
  <option value="freelance">Freelance Project</option>
</select>
```

<label> + for

Ties descriptive text to a control for accessibility — clicking the label text focuses the matching input. The **for** value must match the input's **id**.

```
<label for="em">Email</label>
<input id="em" type="email" name="email">
```

<fieldset> <legend>

Groups a set of related controls inside a labeled box.

```
<fieldset>
  <legend>Contact Details</legend>
  ...
</fieldset>
```

required

An HTML5 attribute that makes the browser block submission until the field is filled in — no JavaScript needed.

```
<input type="text" name="fullname" required>
```

2 Procedure

Work in the same `index.html` file created in the previous experiment. Do not delete the heading, paragraphs, or lists already written — they will be moved inside the new structural tags, and enriched, not replaced.

Part A: Build the Header and Navigation Bar

At the very top of `<body>`, open a `<header>`. Move your existing `<h1>` (your name) inside it, and add a one-line tagline below it in a `<p>`. Then, still inside the header, add a `<nav>` containing one link per section of the page you are about to build (About, Skills, Projects, Achievements, Contact) — each link's `href` should point to a `#id` you will create in Part B, matching exactly.

Part B: Wrap Existing Content in Semantic Sections

Right after `</header>`, open a `<main>` — everything else you add will go inside it, closing it just before your footer. Inside `<main>`, wrap your About paragraph(s) in a `<section>` with `id="about"`, your skills list in a section with `id="skills"`, and your achievements list in a section with `id="achievements"`. Give each section its own `<h2>` heading that matches its purpose.

Part C: Turn Projects into Self-Contained Articles

For the Projects section, replace each plain `<li>` with an `<article>`, since each individual project is a self-contained piece of content that could make sense on its own. Give each article an `<h3>` title and a short `<p>` description.

Part D: Add a Profile Photo

Create a folder named `images` in the same location as `index.html` and place a photo inside it (any `.jpg` or `.png`, keep the file size small). Inside your `<header>`, add an `<img>` pointing to that file, with a meaningful `alt` description, a sensible `width`, and `loading="lazy"`.

Part E: Enrich Your Bio with Text-Level Semantics

Go back to your About paragraph and strengthen its wording using the tags from Section 1.2: wrap your most important skill or achievement in `<strong>`, use `<em>` for a word you want stressed, and wrap any acronym you use (e.g. your degree, or a technology name) in `<abbr>` with a `title` giving its full form. If you have a personal motto or a short quote you like, add it as a `<blockquote>` with a `cite` attribute.

Part F: Upgrade Your Skills List

Convert your flat skills `<ul>` into a nested list grouped by category — for example, a top-level item “Frontend” containing its own `<ul>` of specific skills, and similarly for “Backend” or “Tools.” If you have ever swapped one technology for another while learning, show that evolution on one line using `<del>` for the old skill and `<ins>` for the new one.

Part G: Build a Skills Table

Below your Skills section, add a `<table>` that presents the same information a different way: one column for the skill name, one for its category, and one for your proficiency level (Beginner / Intermediate / Advanced). Structure it with `<thead>` for the column headings and `<tbody>` for the rows. Use `scope="col"` on each header cell. If two or more skills share the same category, merge that category cell across those rows using `rowspan` instead of repeating the text.

Part H: Add External and Download Links

Under each project article, add a link to that project's GitHub repository (or a placeholder link), opening in a new tab with `target="_blank"` and `rel="noopener"`. Separately, if you have a resume file, add a "Download Resume" link elsewhere on the page using the `download` attribute so it saves the file instead of navigating to it.

Part I: Add a Contact Form

Add a new `<section>` with `id="contact"` containing a `<form>`. Group the form's fields inside a `<fieldset>` with a `<legend>` such as "Get In Touch." Include, each with a properly associated `<label>`: a text input for the visitor's name (**required**), an email input for their address (**required**), a `<select>` for their reason for contacting you (e.g. Job Opportunity, Freelance Project, General Enquiry), and a `<textarea>` for their message. Finish with a submit button.

Part J: Add the Footer

Close `</main>`, then add a `<footer>` just before `</body>`. Inside it, wrap your contact details (email, and links to your GitHub/LinkedIn profiles) in an `<address>` tag, followed by a copyright line.

Part K: Test and Review

Save `index.html` and open it in the browser. Click every navigation link and confirm it jumps to the correct section. Click the GitHub/LinkedIn links and confirm they open in a new tab; click the download link and confirm the file downloads; click the email link and confirm the mail app opens. Fill in the contact form and try submitting it empty — confirm the browser blocks submission on the required fields. Finally, use View Page Source (or DevTools → Elements) to confirm the full structure — header, nav, main, section, article, table, form, and footer should all be visible and correctly nested.

3 Tag Reference

Tag	Type	Purpose
<code><header></code>	Structural	Introductory content of the page
<code><nav></code>	Structural	Group of navigation links
<code><main></code>	Structural	Central, unique content of the page
<code><section></code>	Structural	Thematic grouping with its own heading
<code><article></code>	Structural	Self-contained block of content
<code><footer></code>	Structural	Closing content of the page
<code></code> / <code></code>	Text	Importance / emphasis
<code><blockquote></code> / <code><q></code>	Text	Long / short quotation
<code><cite></code> / <code><dfn></code>	Text	Names a source / defines a term
<code><abbr></code>	Text	Abbreviation with full-form tooltip
<code><address></code>	Text	Contact information
<code><ins></code> / <code></code>	Text	Inserted / removed content
<code><sup></code> / <code><sub></code>	Text	Superscript / subscript
<code></code>	Hyperlink	Links to a page, section, or email
<code></code>	Media (void)	Embeds an image
<code><figure></code> / <code><figcaption></code>	Media	Groups an image with its caption
<code></code> / <code></code> / <code></code>	List	Ordered / unordered list and its items
<code><dl></code> / <code><dt></code> / <code><dd></code>	List	Definition list, term, and description
<code><table></code> / <code><tr></code> / <code><td></code>	Table	Table, row, and data cell
<code><th></code>	Table	Header cell
<code><thead></code> / <code><tbody></code> / <code><tfoot></code>	Table	Header / body / footer row groups
<code><form></code>	Form	Container that collects and submits data
<code><input></code>	Form	Single-line control (text, email, radio, etc.)
<code><textarea></code>	Form	Multi-line text input
<code><select></code> / <code><option></code>	Form	Dropdown list of choices
<code><label></code>	Form	Accessible caption for a control
<code><fieldset></code> / <code><legend></code>	Form	Groups related controls with a caption
<code>id="..."</code>	Attribute	Unique identifier, used as a link target
<code>class="..."</code>	Attribute	Reusable label for styling groups of elements

4 Precautions

- Do not use more than one `<h1>`, `<header>`, or `<main>` on the same page.
- Every `<img>` must have an `alt` attribute, even if the image is decorative.
- Every `id` value used in a navigation link must exactly match the `id` on its target section (case-sensitive).
- Use `<table>` only for genuinely tabular data — never to position unrelated content on the page.
- Every form control must have an associated `<label>`; never rely on placeholder text alone.
- Keep image file sizes small so the page loads quickly.
- Save the file after every change and refresh the browser to check the effect immediately, rather than making many changes before testing.

Viva Voce Questions

1. What is the difference between `<section>` and `<article>`?
2. Why is `<img>` called a void/self-closing tag?
3. What is the purpose of the `alt` attribute, and who benefits from it besides sighted users?
4. How does an in-page navigation link (`href="#id"`) actually work?
5. What is the practical difference between `<b>` and `<strong>`?
6. What does the `download` attribute do to a link's normal behavior?
7. When would you use `rowspan` instead of repeating a value in every row of a table?
8. Why must every form input have an associated `<label>`?
9. What happens when a required form field is left empty and the form is submitted?
10. What is the difference between `id` and `class`?

Result / Conclusion

The portfolio webpage was successfully enhanced by restructuring its existing content with semantic HTML5 tags (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`), enriching its text with semantic markup (`<strong>`, `<em>`, `<abbr>`, `<blockquote>`), and by adding a profile image, a nested and updated skills list, a skills data table, internal and external hyperlinks, a downloadable resume link, and a working contact form. The page is now a well-structured, accessible, and data-rich portfolio, ready for CSS styling in the next experiment.